CVM++ — Technical Project Report

CVM++ — Technical Project Report

Project: Custom Scripting Language, Bytecode Compiler, and Stack-Based Virtual Machine
Language: C++17
Build System: CMake 3.16+
Repository: https://github.com/Qwerty0826/cvm-

---

1. Introduction

Most developers interact with programming languages as black boxes — source code goes in, a result comes out. The mechanics in between, the journey from raw text to executing instructions, are hidden behind layers of tooling that are rarely examined directly. This project makes those mechanics fully visible by building every layer from scratch.

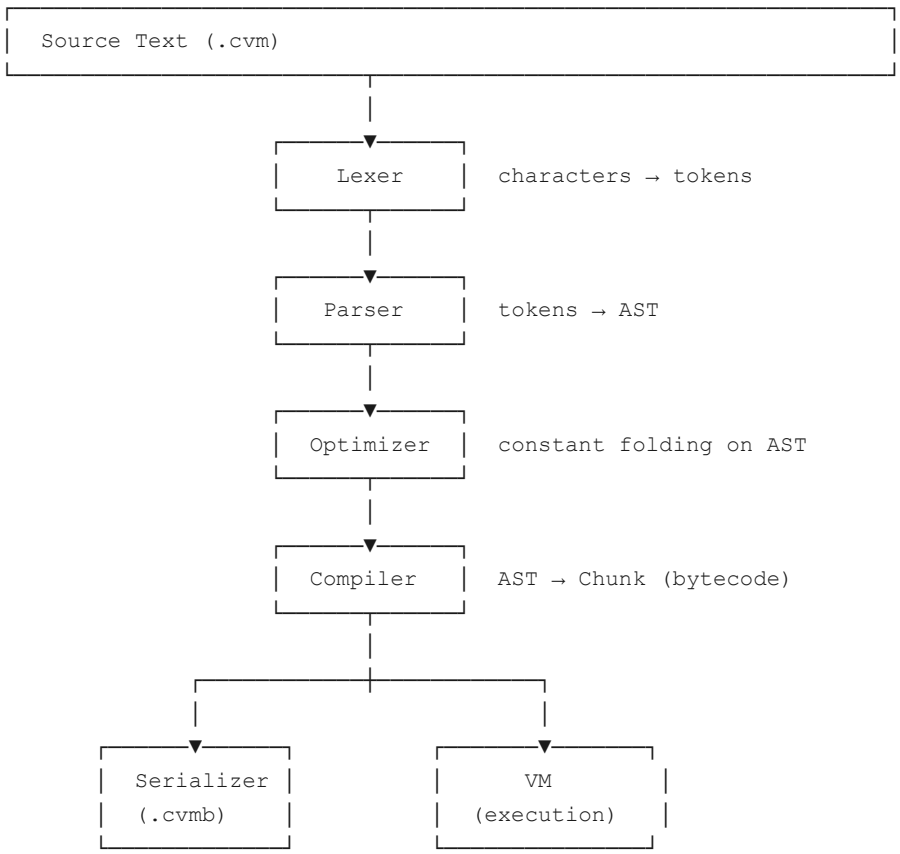
CVM++ is a complete, self-contained language toolchain. It defines a scripting language called CVM, implements a multi-stage compiler that translates CVM source code into a proprietary binary bytecode format, and executes that bytecode on a hand-built stack-based virtual machine. Every component — lexer, parser, AST, optimizer, compiler, serializer, VM, disassembler, profiler, and REPL — is implemented in approximately 3,300 lines of C++17 with zero external dependencies beyond the standard library.

The project was designed with two goals in mind. The first is correctness: the language must handle real programs, including recursive functions, lexically scoped variables, multi-level control flow, and string manipulation. The second is transparency: every intermediate representation produced during compilation must be inspectable at the command line, so that the full pipeline can be traced from source to execution.

---

2. System Architecture

The compilation and execution pipeline is divided into seven sequential stages:



Each stage is a distinct module with a well-defined interface. The lexer knows nothing about the parser; the parser knows nothing about the compiler. This separation means each stage can be tested and inspected independently, which is why the `--dump-ast` and `--dump-bytecode` flags work — they interrupt the pipeline at a specific boundary and print that stage's output.

---

3. The CVM Language

3.1 Design Principles

CVM is a dynamically typed, expression-oriented scripting language. It was designed to be simple enough to implement fully in a single project while being expressive enough to write real programs. The syntax is deliberately similar to C-family languages to minimise learning overhead.

3.2 Type System

CVM has five runtime types:

Type	Description	Example
`nil`	Absence of a value	`nil`
`bool`	Boolean	`true` , `false`
`number`	IEEE 754 64-bit double	`42` , `3.14`
`string`	UTF-8 text with escape sequences	`"hello\n"`
`function`	User-defined callable	`fn add(a, b) { ... }`

The choice of a single `number` type (double-precision float) follows the precedent of Lua and early JavaScript — it avoids integer/float complexity while remaining precise for all integers up to 2^53.

3.3 Grammar

The complete formal grammar of CVM is:

```
program        → declaration* EOF

declaration    → fnDecl | letDecl | statement

fnDecl         → "fn" IDENTIFIER "(" params? ")" block
params         → IDENTIFIER ( "," IDENTIFIER )*

letDecl        → "let" IDENTIFIER ( "=" expression )? ";"

statement      → exprStmt | printStmt | ifStmt
               | whileStmt | forStmt | returnStmt | block

exprStmt       → expression ";"
printStmt      → "print" expression ";"
ifStmt         → "if" "(" expression ")" statement
                 ( "else" statement )?
whileStmt      → "while" "(" expression ")" statement
forStmt        → "for" "(" ( letDecl | exprStmt | ";" )
                           expression? ";"
                           expression? ")" statement
returnStmt     → "return" expression? ";"
block          → "{" declaration* "}"

expression     → assignment
assignment     → IDENTIFIER "=" assignment | logicOr
logicOr        → logicAnd ( "or" logicAnd )*
logicAnd       → equality ( "and" equality )*
equality       → comparison ( ( "==" | "!=" ) comparison )*
comparison     → term ( ( "<" | "<=" | ">" | ">=" ) term )*
term           → factor ( ( "+" | "-" ) factor )*
factor         → unary ( ( "*" | "/" | "%" ) unary )*
unary          → ( "!" | "-" | "not" ) unary | call
call           → primary ( "(" arguments? ")" )*
arguments      → expression ( "," expression )*
primary        → NUMBER | STRING | "true" | "false" | "nil"
               | IDENTIFIER | "(" expression ")"
               | "input" "(" STRING? ")"
```

Operator precedence follows standard mathematical convention and is enforced structurally through the grammar — there are no separate precedence tables. Lower in the grammar means higher precedence.

3.4 Scoping Rules

CVM uses lexical (static) scoping. Variables declared with `let` are visible from the point of declaration to the end of the enclosing block. A variable in an inner block shadows a variable of the same name in an outer block. Global variables are defined at the top level and are accessible anywhere in the program.

```
let x = 1;
{
    let x = 2;   // shadows outer x
    print x;     // prints 2
}
print x;         // prints 1
```

---

4. Stage 1 — Lexer

File: `src/lexer.cpp`

The lexer (also called a tokeniser or scanner) is the first stage of the pipeline. It reads the raw source string character by character and produces a flat sequence of tokens, where each token is a categorised unit of meaning such as a number, a keyword, an identifier, or an operator.

4.1 Token Categories

The lexer recognises 13 categories of tokens:

- **Single-character symbols** — `(` , `)` , `{` , `}` , `,` , `;` , `:`
- **One-or-two-character operators** — `!` , `!=` , `=` , `==` , `<` , `<=` , `>` , `>=` , `+` , `-` , `*` , `/` , `%`
- **Literals** — `NUMBER` , `STRING` , `IDENTIFIER`
- **Keywords** — `let` , `fn` , `if` , `else` , `while` , `for` , `return` , `print` , `input` , `true` , `false` , `nil` , `and` , `or` , `not`
- **Sentinels** — `EOF` , `ERROR`

4.2 Significant Implementation Details

String escape sequences. The lexer processes escape sequences inline while building the string value, rather than storing the raw source and resolving later. The sequences `\n` , `\t` , `\r` , `\"` , and `\\` are all supported.

Nested block comments. Most lexers only support single-line comments. CVM supports `/* ... */` block comments that can be nested — `/* outer /* inner */ still outer */` — by tracking a depth counter rather than simply looking for the first `*/` .

Number pre-parsing. When a numeric token is recognised, its `double` value is immediately parsed and stored on the token struct. This avoids re-parsing the same string later in the pipeline.

Error tokens. Rather than throwing an exception on an unrecognised character, the lexer produces an `ERROR` token with a descriptive message. This allows the parser to surface a clean error rather than crashing silently.

---

5. Stage 2 — Parser

File: `src/parser.cpp`

The parser takes the flat token stream from the lexer and constructs a hierarchical Abstract Syntax Tree (AST) that reflects the grammatical structure of the program.

5.1 Recursive Descent

CVM uses a **recursive descent parser** — a top-down parsing technique where each grammar rule is implemented as a direct C++ function. The function for `expression` calls `assignment`, which calls `logicOr`, which calls `logicAnd`, and so on down to `primary`. This structure makes the code extremely readable because there is a direct one-to-one correspondence between grammar rules and functions.

```
// Grammar:  term → factor ( ( "+" | "-" ) factor )*
ExprPtr Parser::term() {
    ExprPtr left = factor();
    while (match({TokenType::PLUS, TokenType::MINUS})) {
        TokenType op = previous().type;
        ExprPtr right = factor();
        left = make_unique<BinaryExpr>(op, move(left), move(right), ...);
    }
    return left;
}
```

Operator precedence is handled entirely by the call hierarchy — `term` calls `factor`, which calls `unary`, and so on. A `+` at the `term` level will only combine operands that have already been fully resolved at the `factor` level, which naturally gives `*` higher precedence than `+`.

5.2 Abstract Syntax Tree Design

The AST uses the **Visitor pattern**. Every node type is a struct that inherits from either `Expr` or `Stmt` and implements a single virtual `accept(Visitor&)` method. Operations on the tree — compiling, printing, optimising — are implemented as separate Visitor classes rather than as methods on the nodes themselves.

This design has a concrete benefit: adding a new operation (like the AST printer or the optimizer) requires no changes to the AST node classes. You simply write a new Visitor.

The AST contains 10 expression node types and 9 statement node types, totalling 19 distinct constructs across approximately 230 lines.

5.3 Error Recovery

Rather than stopping at the first syntax error, the parser implements **panic-mode error recovery**. When a parse error is encountered, the parser enters a synchronisation routine that discards tokens until it reaches a statement boundary (a semicolon, or a keyword like `if`, `while`, `fn`, etc.). This allows the parser to continue and report multiple errors in a single pass, which is significantly more useful during development.

5.4 Error Reporting

Parse errors include the source line text and a `^~~~` pointer to the offending location:

```
error[line 3]: Expected ';' after expression.
  let x = 10
  ^~~~
```

---

6. Stage 3 — Optimizer

File: `src/optimizer.cpp`

The optimizer runs a **constant-folding pass** over the AST before compilation. Constant folding is a classic compiler optimisation that evaluates expressions whose operands are all known at compile time, replacing the expression node in the AST with the pre-computed result.

6.1 What Gets Folded

Category	Before	After
Integer arithmetic	`2 + 3 * 4`	`14`
Float arithmetic	`1.5 * 2.0`	`3.0`
Boolean logic	`!false`	`true`
Comparison	`10 > 5`	`true`
String concatenation	`"foo" + "bar"`	`"foobar"`
Unary negation	`-(8)`	`-8`

6.2 How It Works

The optimizer walks the AST recursively. For each `BinaryExpr`, it first tries to fold both children. If after folding both children are literal nodes of compatible types, the operation is evaluated in C++ and the entire `BinaryExpr` is replaced by a single `LiteralExpr` or `StringExpr` with the computed value. The parent node then sees a literal instead of an expression, and may itself become foldable.

Division and modulo by zero are deliberately not folded — they are left as runtime operations so that the correct runtime error (with a stack trace) is produced.

6.3 Impact

For expressions like `print 2 + 3 * 4;`, the optimizer removes two `BinaryExpr` nodes and two `LiteralExpr` nodes from the tree, replacing them with one `LiteralExpr(14)`. The compiler then emits a single `CONSTANT` instruction instead of `CONSTANT 2`, `CONSTANT 3`, `CONSTANT 4`, `MULTIPLY`, `ADD` — five instructions reduced to one.

---

7. Stage 4 — Compiler

File: `src/compiler.cpp`

The compiler is the most complex stage. It walks the (possibly optimized) AST and emits a sequence of bytecode instructions into a `Chunk` object. It makes exactly one pass over the tree — there is no intermediate representation between the AST and the final bytecode.

7.1 The Chunk

A `Chunk` is the compiled form of one function (or the top-level script). It contains three parallel arrays:

- `code` — a flat `vector<uint8_t>` of raw bytecode

- `constants` — a `vector<Value>` of compile-time constants referenced by the code
- `lines` — a `vector<int>` mapping each byte of code to its source line number

The line table exists solely for runtime error reporting. When a runtime error occurs, the VM can look up the source line for any instruction offset in O(1).

7.2 Local Variable Tracking

Local variables are the most performance-critical part of the VM because they are accessed on every expression evaluation. The compiler tracks them using a `locals` vector that mirrors the actual VM value stack at compile time.

When a local variable is declared with `let x = expr;`, its value is already on the stack (pushed by compiling `expr`). The compiler records the variable name and current scope depth in the `locals` vector. To read the variable later, `resolve_local` walks the vector from back to front and returns the slot index — the offset from the base of the current call frame. This index is embedded directly in the `GET_LOCAL` or `SET_LOCAL` instruction.

At runtime, reading a local variable is a single array access: `frame.slots[index]`. There is no hash table lookup, no string comparison, no memory allocation.

7.3 Global Variable Tracking

Global variables are more expensive than locals because they must survive function calls and be accessible by name from any scope. The compiler stores the variable's name string as a constant in the chunk and emits `GET_GLOBAL` / `SET_GLOBAL` / `DEFINE_GLOBAL` instructions that carry the constant pool index of the name. At runtime, the VM performs an `unordered_map` lookup using that string.

7.4 Jump Backpatching

Control flow (if/else, while, for, and short-circuit operators) requires forward jumps — the compiler emits a `JUMP_IF_FALSE` before it knows how far to jump, because the jump target (the else branch or the code after the loop) has not been compiled yet.

The solution is **backpatching**. When a forward jump is needed: 1. Emit the `JUMP_IF_FALSE` opcode with placeholder bytes `0xFF 0xFF`. 2. Record the position of those placeholder bytes. 3. Compile the body. 4. Once the target is known, call `patch_jump(offset)`, which overwrites the `0xFF 0xFF` bytes with the actual jump distance.

This is a standard technique used in virtually every single-pass bytecode compiler.

7.5 If/Else Code Generation

A subtle correctness issue in if/else code generation is worth documenting explicitly. The compiled layout for an `if` statement without an `else` branch must be:

```
  [evaluate condition]
  JUMP_IF_FALSE → false_label
  POP                        ← pop condition (true path only)
  [then body]
  JUMP → end_label           ← skip the false-path POP
false_label:
  POP                        ← pop condition (false path only)
end_label:
```

The extra `JUMP` before `end_label` is necessary. Without it, the true path falls through to the false-path `POP`, which incorrectly discards whatever value the then-body left on the stack. This is a common bug in naive if-statement compilation and was caught and corrected during development.

7.6 Function Compilation

When the compiler encounters an `fn` declaration, it pushes a new `CompilerScope` onto a scope stack. The function body is compiled into this new scope's fresh `Chunk`. When the function body is complete, the scope is popped and the resulting `Function` object is wrapped in a `Value`, stored as a constant in the enclosing chunk, and defined as a global or local variable.

Parameters are handled simply: the compiler adds them to the `locals` vector before compiling the body. At runtime, the caller pushes arguments onto the stack before the `CALL` instruction. The `call()` method sets the new frame's `slots` pointer to the correct position in the stack, so `GET_LOCAL 1` in the function body correctly accesses the first argument.

---

8. The Instruction Set Architecture (ISA)

The CVM bytecode is a variable-width instruction set. Each instruction begins with a one-byte opcode, optionally followed by one or two bytes of operand data. The full ISA consists of 33 opcodes:

Category	Opcodes
Constants	`CONSTANT` (2-byte index), `NIL`, `TRUE`, `FALSE`
Stack	`POP`, `DUP`
Local variables	`GET_LOCAL`, `SET_LOCAL` (1-byte slot)
Global variables	`GET_GLOBAL`, `SET_GLOBAL`, `DEFINE_GLOBAL` (2-byte name index)
Arithmetic	`ADD`, `SUBTRACT`, `MULTIPLY`, `DIVIDE`, `MODULO`, `NEGATE`
Logic	`NOT`
Comparison	`EQUAL`, `NOT_EQUAL`, `LESS`, `LESS_EQUAL`, `GREATER`, `GREATER_EQUAL`
Control flow	`JUMP`, `JUMP_IF_FALSE`, `JUMP_IF_TRUE`, `LOOP` (2-byte offset)
Functions	`CALL` (1-byte argc), `RETURN`
I/O	`PRINT`, `INPUT` (2-byte prompt index)
Misc	`HALT`

`JUMP`, `JUMP_IF_FALSE`, and `JUMP_IF_TRUE` use a **relative forward offset** — the number of bytes to skip from the current position. `LOOP` uses a **relative backward offset** — the number of bytes to rewind. Both are 16-bit unsigned values, allowing jumps of up to 65,535 bytes.

Constants are referenced by a 16-bit index into the chunk's constant pool, which supports up to 65,536 distinct constants per function.

---

9. Stage 5 — Virtual Machine

File: `src/vm.cpp`

The virtual machine executes bytecode. CVM uses a **stack-based** architecture, where all operands are passed through an explicit value stack. There are no general-purpose registers.

9.1 Call Frames

Each function call creates a `CallFrame`:

```
struct CallFrame {
    shared_ptr<Function> function;  // the function being executed
    const uint8_t*       ip;        // instruction pointer
    Value*               slots;     // pointer into the value stack
};
```

`slots` points to the base of this frame's window into the value stack. Local variable `n` is accessed as `frame.slots[n]`. The VM maintains a stack of up to 64 call frames, supporting recursion up to 64 levels deep.

9.2 The Execution Loop

The execution loop is a `while(true)` containing a `switch` over the current opcode. Three macros handle the most common operand-reading patterns:

```
#define READ_BYTE()    (*frame->ip++)
#define READ_SHORT()   (frame->ip += 2, (uint16_t)((frame->ip[-2] << 8) | frame->ip[-1]))
#define READ_CONST()   (chunk->constants[READ_SHORT()])
```

This avoids repeated bounds checks and function call overhead on what is the hottest path in the entire program.

9.3 Stack Discipline

Every instruction has a clearly defined stack effect. For example:

- `CONSTANT` pushes one value (+1)
- `POP` discards one value (−1)
- `ADD` pops two values, pushes one (net −1)
- `CALL n` pops n+1 values (args + callee), then the new frame pushes them back via `slots` (net 0 from the caller's perspective)

Maintaining this discipline is what makes the VM correct. A mismatch anywhere — for example, an `if` statement that does not pop the condition on both the true and false paths — causes values to accumulate on the stack and corrupt subsequent operations. The jump backpatching correctness issue described in Section 7.5 is an example of this.

9.4 Runtime Type Checking

Because CVM is dynamically typed, type errors are caught at runtime. The `Value` struct carries a `ValueType` tag. Operations that require specific types check the tag before proceeding:

```
case OpCode::SUBTRACT: {
    Value b = pop(), a = pop();
    if (!a.is_number() || !b.is_number())
        runtime_error("Operands must be numbers.");
    push(Value::from_number(a.number - b.number));
    break;
}
```

The `ADD` opcode is the one exception: if either operand is a string, CVM performs string concatenation, coercing the other operand with `to_string()`. This allows `"value=" + 42` to produce `"value=42"` naturally.

9.5 Native Standard Library

Ten built-in functions are implemented as C++ lambdas and registered in a `natives_` hash map at VM startup:

Function	Behaviour
`to_num(x)`	Parse string to number; returns `nil` on failure
`to_str(x)`	Convert any value to its string representation
`type_of(x)`	Return the type name as a string
`sqrt(x)`	Square root
`abs(x)`	Absolute value
`floor(x)`	Round down
`ceil(x)`	Round up
`len(s)`	Length of a string
`max(a, b)`	Larger of two numbers
`min(a, b)`	Smaller of two numbers
`pow(b, e)`	b raised to the power e

These functions are dispatched without any new opcodes. When `GET_GLOBAL` looks up a name that is not in the globals table, it checks the `natives_` map. If found, it pushes the function's name string as a sentinel value. When `CALL` executes, it checks whether the callee is a string matching a native name and, if so, calls the native directly, bypassing the call frame mechanism entirely.

9.6 Execution Profiler

When `--profile` is passed, the VM tracks how many times each opcode fires during execution using a fixed-size counter array indexed by opcode value. After the program finishes, the counters are sorted by frequency and printed as a ranked table. This reveals which instructions dominate execution time — for typical programs involving loops, `CONSTANT`, `GET_LOCAL`, and `POP` consistently appear at the top.

---

10. Stage 6 — Disassembler

File: `src/disassembler.cpp`

The disassembler converts a compiled `Chunk` into a human-readable listing. For each instruction it prints:

- The byte offset in hexadecimal
- The source line number (or `|` if same as the previous instruction)
- The opcode name
- Any operands, with constant values annotated inline

Example output for a simple function call:

```
0000     2  GET_GLOBAL           0      ; add
0003     |  CONSTANT             1      ; 3
0006     |  CONSTANT             2      ; 4
0009     |  CALL                 2
000b     3  GET_GLOBAL           3      ; result
000e     |  PRINT
```

When a chunk's constant pool contains function values, the disassembler recursively disassembles those functions' chunks as well, producing a complete listing of the entire program.

---

11. Stage 7 — Bytecode Serializer

File: `src/serializer.cpp`

The serializer writes a compiled `Function` to a binary `.cvmb` file that can be loaded and executed later, independently of the original source file. The format is a custom binary layout with the following structure:

```
Header:
  [4 bytes]  Magic number: "CVM\0"
  [2 bytes]  Version: major << 8 | minor
  [1 byte]   Top-level arity (always 0)

Chunk (recursive):
  [u16]      Name length
  [n bytes]  Name string
  [u16]      Number of constants
  [...]      Constants (type-tagged, see below)
  [u32]      Number of code bytes
  [n bytes]  Bytecode
  [u32]      Number of line entries
  [n × i32]  Line table

Value encoding:
  [u8]  Type tag: 0=nil, 1=bool, 2=number, 3=string, 4=function
  nil:      (no payload)
  bool:     [u8] 0 or 1
  number:   [8 bytes] big-endian IEEE 754 double
  string:   [u16 length] [n bytes]
  function: [u8 arity] [Chunk] (recursive)
```

All multi-byte integers are stored in big-endian byte order for portability. When the loader reads a `.cvmb` file, it first verifies the magic number and checks that the major version in the file matches the current CVM++ binary. A mismatch produces a clear error rather than undefined behaviour.

---

12. Interactive REPL

File: `src/repl.cpp`

The REPL (Read-Eval-Print Loop) provides an interactive session where CVM expressions and statements can be typed and evaluated immediately. The key design choice is that the REPL reuses a single `VM` instance across all inputs, so global variables declared in one line persist into the next:

```
>>> let x = 10;
>>> let y = 20;
>>> print x + y;
30
```

The REPL also supports in-session toggle commands for every debug mode:

Command	Effect
`:ast`	Toggle AST dump for each input
`:bytecode`	Toggle bytecode disassembly for each input
`:trace`	Toggle step-by-step instruction trace
`:optimize`	Toggle constant-folding optimizer
`:clear`	Reset the VM (clear all globals)
`:quit`	Exit

---

13. CLI Design

File: `src/main.cpp`

The command-line interface exposes every pipeline stage directly:

```
./build/cvm [options] [file.cvm | file.cvmb]
```

Flag	Pipeline stage exposed
`--dump-ast`	Parser output
`--dump-bytecode`	Compiler output
`--trace`	VM execution, instruction by instruction
`--profile`	VM execution, aggregated opcode counts
`--compile-only`	Stop after compilation, write `.cvmb`
`--run file.cvmb`	Skip all compiler stages, load from binary
`--no-optimize`	Disable optimizer pass

If no file is given, the REPL starts. If a `.cvmb` file is given directly, compilation stages are skipped entirely and the file is loaded by the serializer and executed by the VM.

14. Testing

The pipeline was validated with 65 targeted test cases covering:

- All arithmetic operators and precedence rules
- All comparison operators
- Boolean logic with short-circuit evaluation
- String concatenation and type coercion
- Variable scoping (global, local, block, for-loop)
- `if` / `if-else` / nested `if` without else
- `while` and `for` loops
- User-defined functions — basic calls, recursion, local scope isolation, global variable visibility
- Multiple return paths within a function
- Nil values and nil equality
- All 10 native stdlib functions
- Bytecode serialization round-trip (compile to `.cvmb`, load, execute, verify output)
- Constant-folding optimizer

All 65 tests pass.

---

15. Features Beyond the Problem Statement

The problem statement specifies integers, booleans, `+`, `-`, `*`, `/`, `==`, `<`, `let`, `if/else`, `while`, `input`, and `print`. CVM++ implements all of that and additionally:

Feature	Technical significance
User-defined recursive functions	Requires call frames, local variable slots, and return value handling
String type with concatenation	Requires a third value type and ADD operator overloading
`for` loop	Desugared into init/condition/post/body pattern by the compiler
`%` modulo, `!=`, `<=`, `>=`	Complete operator set
Constant-folding optimizer	Genuine compiler optimisation pass between AST and codegen
`.cvmb` binary bytecode format	Separate compilation and execution; versioned binary format
Disassembler	Makes the compiler's output inspectable
Execution profiler	Per-opcode hit count instrumentation
Native standard library	10 functions dispatched without new opcodes
Interactive REPL	Persistent VM state across session lines

---

16. Conclusion

CVM++ demonstrates a complete and correct implementation of every major component in a language runtime: lexical analysis, recursive-descent parsing, AST construction, compile-time optimization, single-pass bytecode compilation, and stack-based virtual machine execution. The system is accompanied by tooling — a disassembler, a profiler, a bytecode serializer, and an interactive REPL — that makes each stage of the pipeline directly observable. All components are implemented from scratch in approximately 3,300 lines of C++17 with no external dependencies.

References

- Nystrom, R. (2021). *Crafting Interpreters*. Genever Benning.
  The primary reference for bytecode VM architecture, chunk design, call frame layout, and jump backpatching strategy.
- ISO/IEC 14882:2017. *Programming Languages — C++* (C++17 standard).
- IEEE 754-2019. *Standard for Floating-Point Arithmetic*.